

# Talk Track

Runs ~10 min + questions    Open alongside the artifact    If time is short cut beats 3 and 7

---

## Open with this

“This week I got a model to play the games for the first time. It lost to a random agent. I know exactly why, and it's a fixable reason — that's the week.”

Then share the artifact link and scroll it as you go. Everything below maps to a section of that page.

## The beats

---

0:00–1:00

### Why three builds, and none of them "make it good"

- Three builds this week, each with one job.
- None of them was to make the agent play well. You can't improve what you can't measure honestly.
- Build 1 proved the plumbing. Build 2 tested how we work. Build 3 measured a real model.

1:00–2:00

### Build 1 — the random agent

- An agent that picks legal moves at random, submitted to the real competition.
- The score was never the point. It proves a notebook can install the game engine with no internet, play 110 hidden games, and produce a

valid result file.

- It scored 0.07 out of 100. Everything downstream now has a floor to beat.

Numbers: 0.07 / 100 · 110 hidden games · no internet allowed

---

2:00–3:30

### Build 2 — testing the way we work (Genesis)

- Same code, run through the Genesis harness, to answer: does this way of working earn its place?
- I wrote four tests *before* running it, so the verdict couldn't be fitted afterwards.
- The one that mattered: I changed one line of code, and it refused to mark the work finished until the checks were re-run.
- All four passed, in 16 minutes. It later caught two of my own mistakes.

If asked what it costs: about 16 minutes of overhead, and it blocked a broken notebook from being pushed

---

3:30–5:30

### Build 3 — E0, the first real measurement

- A frozen open-weight model, Gemma-4-E4B, playing 17 held-back games, five times each.
- Before it played anything, I fixed the things that could be fudged later: which games are held back, how many attempts, the sampling settings.
- It lost to random on every measure, and beat random on zero of sixteen games.
- Its replies were all valid. It wasn't confused about how to answer. It chose badly.

Numbers: levels 0.00 vs 0.04 · screens explored 67.5 vs 78.7 · 0/16 games · 0% unreadable replies

---

5:30–7:30

### The diagnosis — the part worth their attention

- I guessed twice and was wrong twice, and both were cheap to check on my laptop with no GPU.
- Guess one: it repeats one move. Guess two: it flips between two moves. Both of those explore *more* than the model did.
- So I added logging of every move it chose, and ran it again. That answered it.
- Split the games by type: where it only presses keys, it matches random exactly. Where it can click, it loses — that's the entire gap.
- It clicks the same dead squares over and over. Corners and edges, where nothing happens.
- A 64×64 grid of digits gives it no good way to decide *where* to click. That's fixable, and it's the next thing I do.

Numbers: keyboard games +0.3 · click games -17.3 · worst: su15 -66.2, sb26 -48.8

---

7:30–9:00

### How we work now — the habit worth stealing

- Five steps, same every time: write it down first, fix the unknowns, rehearse locally, gate then run, record rather than remember.
- Step one is the one to steal: write down what result would prove you wrong, *before* you run anything.
- It turns an opinion into something that can fail, and it makes a negative result publishable instead of embarrassing.
- That's why this week's bad result is still usable: the promises were dated and committed before the data existed.

---

9:00–10:00

### Rules, repo, and the ask

- The repo is private, and that's a rules thing, not secrecy.
- Rule 3.6: competition code can't be shared privately outside your team. Public sharing has to go through Kaggle itself.
- Rule 2.4.b: the game data can't be redistributed, so it's excluded from git.

- Rule 2.5.a: prize winners must open-source code, model and weights — that's why every model I shortlisted is Apache 2.0.
- If anyone wants to work on this with me, the clean way is to join the Kaggle team. Up to 8 people, and merges close 26 October.
- Anything about method I can share freely, and the official starter kit is public if you want to run build 1 yourself in ten minutes.

## Numbers, in one place

|                                                                     |                                                     |                                                                        |
|---------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------------------------|
| <b>0.07</b><br>Random agent's score on the hidden games, out of 100 | <b>0 / 16</b><br>Games where the model beat random  | <b>-17.3</b><br>Model vs random on click games (screens per 100 moves) |
| <b>+0.3</b><br>Model vs random on keyboard-only games               | <b>1.35 h</b><br>GPU used this week, of 30 per week | <b>11.04</b><br>Current leaderboard top score, for scale               |
| <b>30 Sep</b><br>Optional milestone; needs a public notebook        | <b>2 Nov</b><br>Final submission                    |                                                                        |

## Questions they're likely to ask

Isn't a model losing to random just a broken setup?

That was my first worry, which is why I checked before drawing conclusions. Its replies were all valid, and on keyboard-only games it matched random exactly. A broken setup would fail everywhere, not only where clicking is possible.

Why not send it a picture of the screen instead of digits?

Two of the four models I'm comparing are text-only, so a picture would make the comparison unfair. That said, the June winners all used vision, and it's the obvious upgrade once the size comparison is done.

---

Why this model? It's small.

Deliberate. A small model makes the effect of any wrapper easy to see. The big models, 27 to 31 billion, are queued next — that's what the June winners used, and it's where the leaderboard scores of 5 to 11 live.

---

How much does a run cost?

This one was 71 minutes and 1.07 GPU-hours. The big models will be 3 to 4 hours each. The weekly allowance is 30 hours, and it just reset.

---

What's the actual experiment you're building towards?

Whether an agent that remembers its failures and avoids returning to states it has already seen does better than one that doesn't. That came out of the literature review. E0 is the baseline it gets compared against.

---

Can I see the code?

Not privately — rule 3.6 forbids it. Join the Kaggle team and I can share everything, or grab the public starter kit and I'll walk you through it.

---

Why bother with Genesis instead of just coding?

I tested that rather than assumed it. Four tests written in advance; the deciding one was that it refuses to call work finished when the code has changed since the checks ran. It also blocked a broken notebook from being pushed this week.

---

What's your score going to be?

Unknown, and I'd rather not promise. The top of the leaderboard is 11 out of 100, so nobody has solved this. What I can promise is a clean measurement and an honest write-up of what worked and what didn't.

---

## If you're asked something you don't know

**Say so, then say where it would be written down.** Every decision and result this week is recorded in the repo with its source, so “I don't remember, but it's in the register” is a real answer, not a dodge.

**Don't defend the result.** The model losing to random is a measurement, not a setback. The useful sentence is: “that's what it did, here's why, here's the fix.”

**Two things genuinely open right now:** whether to run the second small model before or after fixing the clicking, and how to present clickable positions to the model. Both are fair to put back to the room.

---

Companion to *Random, Genesis, E0*. Figures from this week's runs: Kaggle submission 56171083, kernels `arc-agi-3-e0-gemma4-e4b` and `arc-agi-3-e0-diag`. 12 September 2026.